

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANA SANGAMA, BELAGAVI – 590018**



A Synopsis on

**“QoS-Aware Network-on-Chip (NoC) Router Design and
Verification”**

Submitted for the Major Project Work-Phase 1

(Academic Year 2025-26)

Submitted by

**Syed Mannan Saood (1NT23VL060)
Shyam Sundar Das C V (1NT23VL054)
Nandan S R (1NT23VL031)**

Under the Guidance of

Dr. Murthy M
Assistant Professor, Dept. of VLSI Design & Technology

Department of Electronics Engineering (VLSI Design and Technology)



**An Autonomous Institution Affiliated to Visvesvaraya Technological University. Approved by
UGC/AICTE/Govt. of Karnataka, Accredited by NAAC (Grade 'A+') Bangalore-560064,
Karnataka, INDIA**

Academic Year: 2025–2026

Declaration by Students

We, the undersigned students **Syed Mannan Saood (1NT23VL060)**, **Shyam Sundar Das C V (1NT23VL054)** and **Nandan S R (1NT23VL031)** hereby declare that we are fully aware of the **project conduction procedures** and **evaluation rubrics** as prescribed by the department. We understand that the **project work will be evaluated through continuous assessment** across the defined phases.

<i>Name of the Student</i>	<i>Signature</i>
Student 1	
Student 2	
Student 3	

Confirmation by Guide

I,, hereby declare that I shall guide the above-mentioned students in the project entitled **“QoS-Aware Network-on-Chip (NoC) Router Design and Verification”** and will ensure that the project work is carried out under my supervision and submitted as per the academic requirements for the academic year **2025–2026**.

Table of Contents

Table of Contents.....	3
1. Introduction.....	4
2. Objectives of the Project.....	5
3. Literature Survey and Review	6
4. Limitations of the Existing Work (Gap Analysis) and Motivation.....	7
5. Functional Requirements for the Prototype (Semester VI).....	8
6. Proposed Methodology for Building Prototype (SemesterVI)	9
7. GitHub Repository Details.....	10
8. References.....	11
9. Annexures	12
9.1 Annexure -1 Definition of Prototype or Proof-Of-Concept.....	12
9.2 Annexure -2 Definition of Functional Requirements	13

1. Introduction

The proliferation of many-core processor architectures has made the Network-on-Chip (NoC) the primary on-chip communication backbone for modern System-on-Chip (SoC) designs. In systems comprising tens to hundreds of processing cores, the NoC fabric must simultaneously carry heterogeneous traffic classes — coherence request/response messages, bulk memory transfers, and latency-critical real-time data streams — each exhibiting fundamentally different timing and bandwidth demands. Without Quality of Service (QoS) enforcement at the router level, these traffic classes compete equally for shared resources, resulting in priority inversion and unpredictable latency behaviour that violates system-level timing contracts.

Despite significant research into QoS-aware routing algorithms and virtual channel (VC) based arbitration policies, a critical verification gap persists: the majority of published NoC designs are validated through functional simulation alone, without a structured, coverage-driven verification environment that systematically exercises QoS behavioural properties under adversarial multi-class traffic injection patterns. This project addresses this gap through two complementary contributions. First, a QoS-aware 5-port mesh NoC router is designed in RTL (SystemVerilog), incorporating two-VC priority separation per input port and a hierarchical priority arbiter that guarantees high-priority (HP) flit pre-emption over low-priority (LP) flits at every arbitration cycle. Second, a directed SystemVerilog verification environment is constructed to validate that defined QoS properties hold under worst-case congestion scenarios, with quantitative latency and throughput metrics reported per traffic class. The project draws on the architectural precedent established by the OpenPiton manycore research platform (Princeton University), which implements a mesh NoC with traffic prioritisation hooks, and extends it with a formally specified verification methodology targeting QoS correctness.

2. Objectives of the Project

The following specific and measurable objectives are defined for this project, scoped across all three phases of the capstone:

Example:

- a. To design a QoS-aware 5-port mesh NoC router in RTL (SystemVerilog) supporting two Virtual Channel priority classes (High Priority VC0 and Low Priority VC1) with configurable buffer depth and credit-based flow control.
- b. To implement an XY deterministic routing algorithm guaranteeing deadlock-freedom by construction, with quantitative validation that zero misrouted flits occur across a minimum of 10,000 injected test vectors.
- c. To build a directed SystemVerilog verification environment capable of generating adversarial multi-class traffic scenarios and measuring per-VC average latency, throughput, and buffer occupancy statistics per simulation run.
- d. To validate quantitatively that the high-priority VC (VC0) achieves a statistically significant reduction in average flit latency compared to the low-priority VC (VC1) under network load conditions of 70–90% injection rate (target: $\geq 30\%$ latency reduction).

3. Literature Survey and Review

A thorough survey of 28 journal articles, conference papers, and reference texts was conducted, spanning on-chip interconnect architecture, QoS arbitration theory, formal verification methodology, and modern NoC benchmarking frameworks. The primary sources drawn upon are IEEE Transactions on Computers, IEEE Transactions on VLSI Systems, ACM ASPLOS, DAC, and MICRO proceedings. Six representative works are summarised below.

[1] W. J. Dally and B. Towles, “Route Packets, Not Wires: On-Chip Interconnection Networks,” DAC, 2001.

This foundational work formalised the concept of NoC as a structured interconnect paradigm. The authors proposed virtual channel flow control as the primary mechanism for deadlock avoidance, introduced wormhole switching for low-latency flit routing, and analysed the throughput-latency relationship under uniform random traffic. This paper directly informs the VC-based priority architecture adopted in our design. [2] E. Bolotin, I. Cidon, R. Ginosar, and A. Kolodny, “QNoC: QoS Architecture and Design Process for Network on Chip,” Journal of Systems Architecture, vol. 50, no. 2–3, 2004. QNoC introduced a formal QoS service model for NoC routers, defining guaranteed throughput (GT) and best-effort (BE) traffic classes with separate virtual channel pools. The authors demonstrated priority-based arbitration between GT and BE flits, showing measurable latency reduction for GT traffic under congestion. This paper is the closest methodological precedent for our two-VC priority arbiter design. [3] N. Agarwal et al., “GARNET: A Detailed On-Chip Network Model inside a Full-System Simulator,” ISPASS, 2009. GARNET provides a cycle-accurate NoC simulation framework integrated with gem5, modelling per-VC buffers, credit-based flow control, and arbitration logic. The paper validates its model against RTL implementations, establishing a performance baseline methodology. We adopt GARNET’s traffic injection and latency measurement conventions for our testbench metrics. [4] J. Balkind et al., “OpenPiton: An Open Source Manycore Research Framework,” ASPLOS, 2016. OpenPiton implements a scalable tiled manycore SoC with a mesh NoC featuring arbitration logic and traffic prioritisation hooks for coherence traffic. While OpenPiton demonstrates the viability of priority-aware NoC routing in silicon-targeting RTL, it does not provide a structured verification environment targeting

QoS behavioural properties. This gap directly motivates our project. [5] L.-S. Peh and W. J. Dally, “A Delay Model and Speculative Architecture for Pipelined Routers,” HPCA, 2001. This paper introduced speculative arbitration in pipelined NoC routers, enabling clock-cycle-level throughput analysis. The delay model is used in our router pipeline stage planning. [6] B. Wile, J. Goss, and W. Roesner, Comprehensive Functional Verification: The Complete Industry Cycle, Elsevier/Morgan Kaufmann, 2005. This reference text establishes the theoretical basis for coverage-driven verification, defining functional coverage metrics, constrained-random stimulus generation, and self-checking testbench architectures. Our verification methodology for Phase 2 (UVM) is grounded in this framework.

4. Limitations of the Existing Work (Gap Analysis) and Motivation

The literature review reveals four critical gaps that motivate this project. First, existing QoS NoC architectures (QNoC, GARNET) are evaluated exclusively through simulation-level behavioural models; no published work presents a coverage-driven RTL verification environment that formally closes QoS properties against a synthesisable router implementation. Second, the OpenPiton platform, which is the closest 1:1 architectural precedent for a mesh NoC with priority hooks, provides no defined test plan, coverage model, or pass/fail criteria for QoS behavioural correctness. Its arbitration logic has not been subjected to adversarial traffic injection designed to provoke priority inversion corner cases. Third, most NoC verification literature targets structural correctness (deadlock-freedom, livelock-freedom) rather than service-level properties (worst-case HP latency under LP congestion, credit starvation scenarios). The distinction is significant: a router can be functionally correct under nominal traffic while exhibiting QoS violations under adversarial injection patterns. Fourth, no open-source, parameterisable RTL QoS NoC router exists with an accompanying verification environment and documented coverage closure methodology, leaving a reusability gap for academic and industrial adopters. This project directly addresses all four gaps by delivering a verified, parameterisable RTL QoS router with a documented, reproducible verification methodology.

Example:

"Existing systems fail to address dynamic real-time changes in traffic. They also lack vehicle prioritization for emergency vehicles. This limitation motivated us to design a low-cost, AI-enabled system to improve efficiency and public safety."

5. Functional Requirements for the Prototype (Major Project-Phase 1)

List the functional requirements and use cases that the proposed Prototype System is expected to support. Refer to the Annexure section of this document for definitions and illustrative examples of a Prototype and Functional Requirements.

For the Major Project-Phase 1 scope, only a limited set of core functional requirements (primary success use cases) should be implemented. The emphasis should be on demonstrating the key capabilities of the system rather than full-scale feature completion.

Please note that the Faculty Guide will validate the implemented functional requirements during the final evaluation of Major Project-Phase 1.

Functional Requirements Table To Be Filled Up by the Students -

Req ID	Functional Requirement (SMART compliant)	Expected Outcome
FR1	The router shall correctly route flits arriving at any of 5 input ports (N, S, E, W, Local) to the corresponding output port using deterministic XY routing, with zero misrouted flits across a minimum of 10,000 injected test vectors spanning all source-destination pairs.	Simulation log reports 0 routing errors. Destination port matches XY algorithm prediction for every flit injected.
FR2	The router shall maintain two independent Virtual Channel (VC) buffer pools per input port — VC0 (High Priority) and VC1 (Low Priority) — each with configurable depth (default 4 flits), preventing any data interleaving or corruption between priority classes across all simulation runs.	Scoreboard confirms VC0 and VC1 payload data streams are cleanly separated with zero cross-VC data corruption events logged.
FR3	Under congestion at 70–90% injection load, the priority arbiter shall grant output port access to VC0 flits before VC1 flits in every cycle where both VCs simultaneously request the same output port, achieving a measurable average latency reduction of $\geq 30\%$ for VC0 relative to VC1 across 1,000 congestion test vectors.	Per-VC latency histogram report generated by testbench confirms VC0 average latency is statistically lower than VC1. Arbitration grant log shows zero HP-after-LP grant events when both compete simultaneously.

6. Proposed Methodology for Building Prototype (Major Project-Phase 1)

The Phase 1 prototype is a single 5-port QoS-aware NoC router implemented in synthesisable SystemVerilog, validated through a directed testbench in Synopsys VCS. The development follows a bottom-up build strategy with four incremental milestones. to solving the problem, detailing both the **hardware** and **software components**, tools/technologies used, and the **workflow** or **system design**. A **block diagram** or **flowchart** is often included to illustrate the overall architecture.

Key Points to Include-

- **System Architecture:** The router consists of five pipeline stages: (1) Flit Buffer with 2-VC input queues per port using credit-based flow control; (2) Route Computation unit implementing XY routing (East-first, then North); (3) Virtual Channel Allocator assigning VC0/VC1 per packet header priority bit; (4) Priority Switch Arbiter implementing fixed-priority round-robin, VC0 always pre-empts VC1 at arbitration; (5) Crossbar Switch with 5x5 non-blocking interconnect. The testbench wraps the DUT with a Traffic Generator (configurable injection rate and priority mix per port), a Scoreboard (checks routing correctness and VC data integrity), and a Latency Monitor (records per-VC timestamp delta from injection to ejection).

A labeled diagram showing major components (sensors, processors, communication modules, databases, etc.) and how they interact.

- **Workflow / Process Steps:**

Describe the sequence of operations or data flow through the system.

Example: Sensor → Microcontroller → Data Processing → Decision Logic → Output Device.

- **Tools and Software:** RTL is coded in SystemVerilog (IEEE 1800-2017). Simulation is run on Synopsys VCS with DVE waveform viewer for debug. Cadence SimVision / Xcelium is available as an alternative. Verilator is used for fast regression on directed tests before full VCS runs. All source is version-controlled on GitHub.

Mention microcontrollers, sensors, actuators, or communication modules (e.g., Arduino, Raspberry Pi, LoRa, ESP32, etc.).

- **Build Milestones for Phase 1: Milestone 1 — Implement and simulate a 5-port XY router with single VC (no priority), verify routing correctness via directed tests. Milestone 2 — Add dual-VC buffer architecture per port, verify buffer independence via scoreboard. Milestone 3 — Implement fixed-priority arbiter, inject simultaneous VC0+VC1 contention stimulus, verify VC0 always wins arbitration. Milestone 4 — Instrument testbench with latency monitor, run congestion sweep (10%, 50%, 70%, 90% injection load), generate per-VC latency histogram and confirm $\geq 30\%$ latency advantage for VC0.**

Languages, simulation platforms, libraries, and environments (e.g., Python, MATLAB, Embedded C, Proteus, Keil, Android Studio, etc.).

- **Verification Strategy: Phase 1 uses directed SystemVerilog testbenches (no UVM — UVM is Phase 2 scope). Each functional requirement has a corresponding directed test with documented pass/fail criteria. A self-checking scoreboard compares actual DUT outputs against a software reference model of the XY routing algorithm. Simulation is run with VCS +vcs+lic+wait and waveforms are dumped to VCD for post-processing. Coverage metrics (line, toggle) are collected as a baseline for Phase 2 coverage closure.**

If applicable, describe any machine learning, optimization, or signal processing methods.

Example:

7. GitHub Repository Details

GitHub repository will be initialised at project kickoff. The repository will contain: RTL source (rtl/), testbench source (tb/), simulation run scripts (sim/), and a README with build and run instructions. Repository link: [https://github.com/\[team-repo\]/qos-noc-router-](https://github.com/[team-repo]/qos-noc-router-)

verification (to be updated upon creation). The repository will be made public before Phase 1 evaluation. All commits will be tagged by milestone to allow evaluators to trace incremental progress.

8. References

[1] W. J. Dally and B. Towles, "Route Packets, Not Wires: On-Chip Interconnection Networks," in Proc. 38th Design Automation Conf. (DAC), Las Vegas, NV, USA, Jun. 2001, pp. 684–689. [2] E. Bolotin, I. Cidon, R. Ginosar, and A. Kolodny, "QNoC: QoS Architecture and Design Process for Network on Chip," J. Syst. Archit., vol. 50, no. 2–3, pp. 105–128, Feb. 2004. [3] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "GARNET: A Detailed On-Chip Network Model inside a Full-System Simulator," in Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS), Boston, MA, USA, Apr. 2009, pp. 33–42. [4] J. Balkind, M. McKeown, Y. Fu, T. Nguyen, Y. Zhou, A. Lavrov, M. Shahradd, A. Fuchs, S. Payne, X. Liang, M. Matl, and D. Wentzlaff, "OpenPiton: An Open Source Manycore Research Framework," in Proc. 21st Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), Atlanta, GA, USA, Apr. 2016, pp. 217–232. [5] L.-S. Peh and W. J. Dally, "A Delay Model and Speculative Architecture for Pipelined Routers," in Proc. 7th Int. Symp. High-Performance Computer Architecture (HPCA), Monterrey, Mexico, Jan. 2001, pp. 255–266. [6] B. Wile, J. Goss, and W. Roesner, Comprehensive Functional Verification: The Complete Industry Cycle. Burlington, MA, USA: Elsevier/Morgan Kaufmann, 2005. [7] W. J. Dally and B. P. Towles, Principles and Practices of Interconnection Networks. San Francisco, CA, USA: Morgan Kaufmann, 2004. [8] A. Hemani et al., "Network on a Chip: An Architecture for Billion Transistor Era," in Proc. IEEE NorChip Conf., Turku, Finland, Nov. 2000, pp. 166–173. [9] T. Bjerregaard and S. Mahadevan, "A Survey of Research and Practices of Network-on-Chip," ACM Comput. Surv., vol. 38, no. 1, pp. 1–51, Mar. 2006. [10] G. De Micheli and L. Benini, "Networks on Chips: A New SoC Paradigm," Computer, vol. 35, no. 1, pp. 70–78, Jan. 2002.

Example (IEEE Style):

- [1].J. Smith, *Intelligent Transportation Systems*, McGraw-Hill, 2020.
- [2].A. Sharma and P. Verma, "Adaptive Traffic Signal Control using ML," *IEEE Access*, vol. 9, pp. 45678–45689, 2021.

9. Annexures

9.1 Annexure -1 Definition of Prototype or Proof-Of-Concept

A **Prototype / Proof-of-Concept (PoC)** in the 6th semester is an early demonstration of the technical feasibility of the chosen solution, without aiming for full functionality or production-level quality.

A valid PoC must satisfy the following-

A. Demonstrates Core Feasibility

The PoC must implement one or more core functional requirements to prove that the proposed approach, technology stack, and architecture are viable.

B. Limited Scope, Not Complete Coverage

The PoC is not expected to implement all functional or any of non-functional requirements; it may use assumptions, mock data, simplified workflows, or partial integrations.

C. Exploratory by Design

The PoC is intended to validate assumptions, identify risks, and surface technical challenges, even if parts of the solution fail or need redesign.

D. Architecture-Guiding Artifact

Successful PoC implementation and demonstration should influence the final system architecture, technology choices, and requirement refinement in the 7th semester.

E. Clearly Distinguishable from the Final Project

The PoC prioritizes learning and feasibility over robustness, optimization, scalability, security, or polish, all of which are addressed systematically in later semesters.

9.2 Annexure -2 Definition of Functional Requirements

What Are Functional Requirements? Functional Requirements describe **WHAT your system must do.** They define the features, behaviors, and capabilities that your prototype or proof-of-concept must demonstrate. Examples: • A user can log in • System captures sensor data • Model classifies images • Dashboard displays results

Functional Requirements must be SMART compliant. Writing Good Functional Requirements -> {Actor (Who is the User?) + Action (what action?) + System Response (expected response from the system)}. Avoid mentioning underlying technology details in functional requirements.

Example: The system shall allow users to reset their password via email verification.

Each requirement should be -

- **S – Specific** → Clearly defined
- **M – Measurable** → Can be validated/tested
- **A – Achievable** → Realistic for prototype
- **R – Relevant** → Directly tied to problem
- **T – Time-Bound** → Feasible within Semester VI

Example Entries (SMART Compliant Requirements)

Example 1 – Web / Software Project

Req ID	Functional Requirement (SMART)	SMART Justification	Expected Outcome
FR1	The system shall allow users to log in using email and password authentication.	Specific: Login via email/password Measurable: Success/failure validation Achievable: Basic module of Web Projects Relevant: Core functionality Time-Bound: Implemented in Sem VI prototype	Users can securely access the system using valid credentials. Unauthorized users shouldn't be able to access the software

Example 2 – IoT / Embedded Project

Req ID	Functional Requirement (SMART)	SMART Justification	Expected Outcome
FR2	The prototype shall capture temperature data every 5 seconds using the sensor module.	Specific: Temperature capture Measurable: 5-second interval Achievable: Standard sensor setup Relevant: Data acquisition Time-Bound: Implemented in prototype	Continuous real-time temperature data collection is demonstrated.

Example 3 – AI / ML Project

Req ID	Functional Requirement (SMART)	SMART Justification	Expected Outcome
FR3	The system shall classify input images into predefined categories using a trained ML model.	Specific: Image classification Measurable: Prediction accuracy Achievable: Pretrained/basic model Relevant: Core AI function Time-Bound: Prototype-level implementation	The model predicts image categories with observable accuracy.